Лабораторная работа 13. Point-in-time признаки и Feature Store

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Продолжительность: 2 академических часа и самостоятельная работа Предшествующая работа: ЛР12 «Drift и отложенное качество модели»

1. Цель

Сформировать исторически корректную выборку признаков, доказать утечку в наивном latest join, учесть available time и TTL, а затем связать contract с демонстрационным Feast repository.

2. Результаты обучения

Каждый студент должен уметь:

- объяснить entity, event time, feature time и available time;
- воспроизвести наивный и point-in-time join;
- доказать отсутствие будущих значений;
- интерпретировать feature age и coverage;
- диагностировать нарушение TTL или ключа;
- объяснить offline и online retrieval;
- связать feature service с версией модели;
- защитить личный тест, contract или audit.

3. Командный результат

Команда предоставляет tag `lr13-v1`:

1. `features/feature_contract.json`;
2. entity events и feature history с UTC timestamps;
3. `naive.csv` и `point-in-time.csv`;
4. `audit.json` с leakage, coverage и feature age;
5. минимум четыре автоматических теста;
6. определения Feast как необязательное расширение;
7. документацию временной семантики;
8. общий baseline и отдельный audit каждого участника;
9. issue, PR, review и индивидуальные evidence cards.

4. Проверенная версия расширения

Обязательная реализация использует pandas. Feast допускается для демонстрации в версии `>=0.67,<0.68`; stable 0.67.0 проверена 20 октября 2026 года. Установка Feast и запуск online store не являются условием допуска. Скриншот UI не заменяет definitions, тест и audit JSON.

5. Подготовка

```
python -m pip install -e ".[dev]"
python -m pytest tests/test_features.py -q
```

Дополнительное Feast-окружение:

```
python -m pip install -e ".[feature_store]"
```

6. Учебная схема

Entity events:

Поле	Смысл
`request_id`	уникальная строка решения
`airport_id`	entity join key
`event_time`	момент, для которого строится прогноз
`ArrDelay`	отложенная цель, не признак

Feature history содержит `airport_id`, `feature_time`, `available_time`, `trailing_delay_rate`, `flights_last_24h`.

7. Feature contract

Contract фиксирует version, entity key, три timestamp-семантики, список признаков, TTL, допустимую долю пропусков и владельца. Изменение units или смысла признака требует новой версии.

8. Проверки до join

Проверьте обязательные столбцы, UTC timestamps, уникальность `request_id`, тип entity key и временную идентичность feature history. Недетерминированные дубликаты отклоняются до join.

9. Наивный baseline

`naive_latest_join` выбирает последнюю строку истории для каждого аэропорта без учёта event time. Это намеренно ошибочный пример. Сохраните его результат и подсчитайте строки, где feature time или available time позже события.

10. Point-in-time join

Для каждого события оставьте feature rows той же entity, которые удовлетворяют:

- `feature_time <= event_time`;
- `available_time <= event_time`;
- возраст неотрицателен и не превышает TTL.

Выберите последнюю допустимую строку. Результат содержит одну строку на `request_id`.

11. Available time fixture

В истории аэропорта A запись относится к 07:00, но доступна только в 09:00. Для события в 08:00 корректный join обязан выбрать более раннее значение 0.35, а не 0.55. Этот fixture обязателен: он доказывает, почему одного feature time недостаточно.

12. TTL

Базовый TTL равен 48 часам. Измените его в личной ветке и покажите влияние на coverage. Нельзя увеличивать TTL только ради заполнения всех строк без анализа допустимой устарелости.

13. Базовый запуск

```
python -m mlops_flight_delay.features \
--entities data/features/entities.csv \
--history data/features/history.csv \
--contract features/feature_contract.json \
--output-dir reports/features/lr13-team-baseline
```

Ожидаемый audit: шесть entity rows, leakage в naive join, ноль leakage в point-in-time join, coverage 1.0 и status `pass`.

14. Audit JSON

Отчёт содержит contract version, число строк, `naive_leakage_rows`, `point_in_time_leakage_rows`, `rows_changed_vs_naive`, coverage, maximum feature age и status. Команда не удаляет наивный результат: он является проверяемым отрицательным примером.

15. Coverage

Coverage равен доле событий, для которых найдены все обязательные признаки. Укажите числитель и знаменатель в пояснении. При пропуске исследуйте entity mismatch, TTL, available time и источник. Молчаливое заполнение средним не принимается.

16. Feature age

Для выбранной строки сохраните возраст в часах. Отрицательный возраст означает leakage, возраст выше TTL — stale feature. Максимум полезен как evidence, но распределение возрастов информативнее одного числа; участник может добавить histogram как личный артефакт.

17. Training-serving parity

Зафиксируйте имя, тип, units и transformation каждого признака. Покажите, что inference ожидает тот же набор и contract version. Если online value сравнивается с historical, укажите сопоставимый момент и допустимую freshness.

18. Feast demo

Каталог `feature_repo/` содержит entity `airport`, feature view `airport_stats` и feature service `flight_delay_v1`. Перед `feast apply` преобразуйте CSV в Parquet. Для historical retrieval передайте entity dataframe с `airport_id` и timestamp. Сравните результат с pandas point-in-time output.

19. Historical и online retrieval

`get_historical_features` используется для training и batch scoring. `get_online_features` получает актуальные значения для inference и не строит историческую выборку. На защите студент объясняет это различие даже если Feast не устанавливался.

20. Отказные сценарии

Каждый участник выбирает собственный сценарий:

Сценарий	Ожидаемый результат
убрать фильтр feature time	audit обнаруживает будущее
убрать available time	fixture в 08:00 выбирается неверно
уменьшить TTL	coverage падает
дублировать request ID	явный отказ контракта
изменить entity key	появляются пропуски или неверный join
использовать online latest для training	расхождение с historical output

Сценарий выполняется в отдельной ветке или копии данных.

21. Автоматические тесты

Обязательны проверки:

1. feature time не позже event time;
2. available time не позже event time;
3. fixture недоступной записи выбирает прошлое значение;
4. naive join содержит leakage;
5. point-in-time join не содержит leakage;
6. duplicate request ID отклоняется;
7. feature age не превышает TTL;
8. JSON parseable и содержит contract version.

22. CI

Workflow запускает весь pytest и CLI-аудит, сохраняет `reports/features/lr13-ci`. Feast не устанавливается в основной job, потому что является расширением. Команда может добавить отдельный manual job с pinned extra, не замедляя обязательный quality gate.

23. Командные роли

Для команды 2–5 человек:

- владелец entity contract и качества входов;
- владелец point-in-time алгоритма;
- владелец leakage fixtures и тестов;
- владелец parity/coverage audit;
- владелец Feast definitions и документации.

В паре роли совмещаются, но у каждого остаётся самостоятельный артефакт. Роли ротируются после ЛР12.

24. Индивидуальная сдача участника команды

Командный результат: tag `lr13-v1`, contract, два join output, audit, тесты и документация.

Каждый участник заполняет `docs/evidence/lr13-evidence-card.md`. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР13 и tag `lr13-v1`
Роль	роль и отличие от ЛР12
Задача	собственная issue с критерием приёмки
Личный артефакт	join, contract, test, fixture, audit, parity или Feast definition
Запуск	личный сценарий и отдельный output directory
Evidence	leakage rows, coverage, feature age и contract version
Решение	временная семантика, альтернатива и риск
Использование ИИ	инструмент и самостоятельно проверенные утверждения

25. Минимальный допуск

1. Командная версия зафиксирована tag и commit SHA.
2. Есть собственная issue и наблюдаемый критерий.
3. Личный артефакт встроен в командную версию.
4. Изменение прошло PR и review другого участника.
5. Личный audit воспроизводим.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент объясняет все три времени и TTL.

26. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag;
2. воспроизводит свой отказной сценарий;
3. находит leakage в naive output;
4. объясняет выбранную строку PIT join;
5. изменяет TTL, время или entity key по заданию преподавателя;
6. интерпретирует coverage и feature age;
7. объясняет различие historical и online retrieval.

27. Повторная сдача и стоп-факторы

Повторная сдача выполняется через переоткрытую issue, отдельный PR, новый audit и обновлённую evidence card. История первой попытки сохраняется.

Стоп-факторы:

- latest join выдан за point-in-time корректный;
- available time отсутствует или игнорируется;
- target того же события используется как feature;
- TTL проверяется только текстом;
- результат имеет дубликаты request ID;
- online latest используется для historical training;
- Feast UI заменяет definitions и тест;
- общий audit выдан за личный;
- студент не может объяснить выбранную строку.

28. Критерии оценивания

Критерий	Баллы	Полное выполнение
Contract и временная семантика	2	entity и три времени определены однозначно
Point-in-time join	2	future, unavailable и stale rows исключены
Leakage audit и coverage	2	naive/correct доказательно сопоставлены
Тесты, CI и parity	2	сценарии воспроизводимы, contract связан с inference
Индивидуальный вклад и защита	2	личный audit, артефакт, review и объяснение

Максимум: 10 баллов. Рекомендуемый порог зачёта — 6 баллов при отсутствии стоп-факторов.

29. Чек-лист

- [ ] Команда состоит из 2–5 человек, роли ротированы.
- [ ] Entity key и timestamps описаны в contract.
- [ ] Request ID уникален.
- [ ] Naive output сохранён как отрицательный пример.
- [ ] Feature time не позже event time.
- [ ] Available time не позже event time.
- [ ] Feature age не превышает TTL.
- [ ] Coverage соответствует policy.
- [ ] Target не используется как feature.
- [ ] Historical и online retrieval разделены.
- [ ] Тесты и CI успешны.
- [ ] У каждого участника отдельный audit и evidence card.
- [ ] Итоговая версия отмечена tag `lr13-v1`.

30. Связь с ЛР14

В ЛР14 feature contract, owner, sources, ограничения, coverage и возможное влияние на сегменты переносятся в model card и data card. Использование внешних данных требует проверки лицензии и правил хранения.

31. Официальные источники

- Feast feature retrieval: https://docs.feast.dev/getting-started/concepts/feature-retrieval

- Feast feature view: https://docs.feast.dev/getting-started/concepts/feature-view
- Feast concepts: https://docs.feast.dev/getting-started/concepts
- Feast PyPI: https://pypi.org/project/feast/